SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Department of Robotics and Automation

Hospital Medicine Delivery System

Comprehensive Project Documentation

Web application, robot firmware, hardware, testing and review findings

B.Tech Robotics and Automation | Academic year 2026-27

Guide: Dr. Mrunal Mishra, Assistant Professor

Dhruv Asarawala 23070127013

Ayush Chhaparia 23070127015

Dhruv Goyal 23070127021

Pranay Trivedi 23070127043

Compiled 24 October 2026 from the README, setup guide, robot firmware (R1 and R2), CA-4 project review report and report brief

1. Project Overview

1.1 Summary

The Hospital Medicine Delivery System connects three activities that hospitals usually handle separately: medicine requisitions from the wards, stock control in the central medicine room, and physical transport of the medicines. Ward staff place orders in a web application and stock is reserved immediately. The medicine-room manager accepts, prepares, loads and dispatches each order. One of two ESP32-based line-following robots then carries the sealed cargo box along a black-line track to the ward. **R1** serves General Ward 1 and General Ward 2, and **R2** is dedicated to the Emergency Ward / Operation Theatre.

At the destination the ward staff unlock the box with an RFID card, remove the medicines and close the lid with a second scan. The robot reports the handover, turns around, drives back to the medicine room, reports its return and becomes available for the next order.

The project has three technical parts:

- **Web application** (Python, Flask, SQLite): role-based pages for the administrator, the medicine-room manager and three wards, with inventory reservation, a priority queue, robot assignment and an audit trail.
- **Robot HTTP API**: token-protected endpoints through which a robot fetches its assignment and reports heartbeat, arrival, handover and return.
- **Robot firmware** (two Arduino sketches, `bot_general.ino` and `bot_emergency.ino`): Wi-Fi client, line following with two IR sensors, RFID access control, servo-locked cargo box, motor control through an L298N and a status display on an ILI9341 TFT.

1.2 Problem and motivation

Manual medicine transport pulls clinical or support staff away from other duties, makes the sequence of outstanding requests hard to see, and gives no single record of what stock has been promised to which ward. Emergency requests need different treatment from routine requests, and two wards can ask for the same units at the same moment. The project addresses the information and coordination part of this process, and prototypes the physical delivery part with a small robot fleet.

> These are the motivations for the project. Reductions in delivery delay, staff workload or clinical risk have **not been measured** for this prototype, and this document does not claim them.

1.3 Project titles

Two titles appear in the project files, and the team should settle on one before final submission:

- **Certificate and report brief:** "Hospital Medicine Delivery System Using Multi-Agent Swarm Robotics and Preemptive Routing".
- **Review report cover:** "Autonomous Medical Swarm Robotics for Intra-Ward Supply Logistics with Priority Scheduling and Vacancy Tracking".

The review report is careful to note that, in the current implementation, "swarm" and "preemptive routing" describe a research direction. The system uses a centralised queue with two robots, and an active mission is never interrupted. "Vacancy tracking" (load-cell based stock counting) is not present in either the web application or the firmware.

1.4 Technology stack

Table 1. Technologies used in the project

Layer	Technology	Notes
Backend	Python 3.10 to 3.12 (README names 3.11), Flask 3.x, Flask-SQLAlchemy, Flask-Login, Flask-WTF, Werkzeug, python-dotenv	Dependencies use lower-bound versions in `requirements.txt` (no lock file)
Database	SQLite, file `instance/hospital.db`	In-memory SQLite is used by the test configuration
Frontend	Jinja templates, HTML5, CSS, vanilla JavaScript (`app.js`, `manager.js`)	Responsive layout; Fetch API for JSON endpoints
Robot link	HTTP/JSON over 2.4 GHz Wi-Fi; `X-Robot-Token` header	Robots poll the server; there is no MQTT or push channel
Controller	ESP32 Dev Module, Arduino IDE 2.x	Firmware supports both Arduino-ESP32 core 2.x and 3.x LEDC APIs
Firmware libraries	WiFi, HTTPClient, SPI, ArduinoJson, Adafruit GFX, Adafruit ILI9341, MFRC522, ESP32Servo	Five libraries need installing; the rest ship with the ESP32 core
Robot hardware	2 IR line sensors, RC522 RFID, SG90-class servo lock, L298N driver, ILI9341 TFT	Models other than the part numbers named in the files are not documented
Testing	pytest	14 tests passed on 22 October 2026 according to the review report

1.5 Implementation status at a glance

The review report was written when no robot firmware had been supplied, so it labelled the physical layer as "future work". The two firmware files now exist, and the table below updates the status accordingly. Hardware trials have still not been recorded.

Table 2. Status of major features (updated to include the firmware)

Feature	Status	Evidence and boundary
Role-based web interfaces and login	Implemented	Source inspection in the review report; twelve screenshots of the interface
Catalog, stock reservation, manual stock reception	Implemented	Inventory service and manager route; backend tests pass
Three distinct ward ranks and FIFO queue	Implemented	Queue key is rank, creation time, then id. Manager actions are not forced to follow the queue
R1/R2 records and manual robot assignment	Implemented	Assignment requires an active, AVAILABLE robot with no current order
Robot HTTP API (server side)	Implemented	Six token-protected routes; covered by an API contract test
Robot firmware: line following, RFID unlock, servo lock, TFT, HTTP client	Firmware written	Two sketches read in full for this document. Not yet validated by recorded bench or track trials
Physical build record (photos, bill of materials, measurements)	To record	Pin map exists in the setup guide. Battery, motors, chassis and wheel data are not documented
Mission progression dispatch to return	Simulated / hardware pending	Can be driven from the browser or by the firmware; no measured physical run on file
Automatic dispatch, nearest-robot or battery-aware assignment	Proposed	Manager chooses the robot manually

Feature	Status	Evidence and boundary
Obstacle avoidance, route planning, active-mission preemption	Future work	No code in the web application or firmware
Swarm (decentralised) behaviour	Not implemented	Control is centralised
Load-cell stock counting ("vacancy tracking")	Future work	Stock is entered manually

1.6 Sources and conventions

This document was compiled from the following project files:

File	What it provided
`README.md`	System description, roles, business rules, robot states, robot API, test scenario list
`SETUP_GUIDE.txt`	Installation steps, Arduino setup, pin map, demo workflow, troubleshooting
`bot_general.ino`, `bot_emergency.ino`	Robot firmware: states, navigation logic, RFID rules, HTTP calls, tunable parameters (read directly)
`CA4_btechProjectReviewReport_final.pdf`	Literature review, problem statement, source-code inspection of the web application, database design, test results, work plan, references
`Medibot.docx`	The report brief: chapter structure and the requirement to keep implemented, simulated, proposed and measured items separate

> The web application source files (`app.py`, `models.py`, routes and services) were **not** supplied for this document. Backend details are taken from the README and from the source inspection recorded in the review report. Firmware details come from reading the two `.ino` files directly.

Status labels used throughout: **Implemented** (code path exists), **Firmware written** (code exists, hardware trial unrecorded), **Simulated** (state can be advanced without proof of physical movement), **Proposed** (design idea only), **Future work**, and **To record / To verify** (information the team still needs to supply).

2. System Architecture

2.1 Overview

Browsers talk to a single Flask application. The application separates route handlers from service modules, and the services work on one SQLite database. The robots are clients of the same application: they poll an authenticated HTTP API and post events. Nothing is pushed to the robots, and robots do not talk to each other.

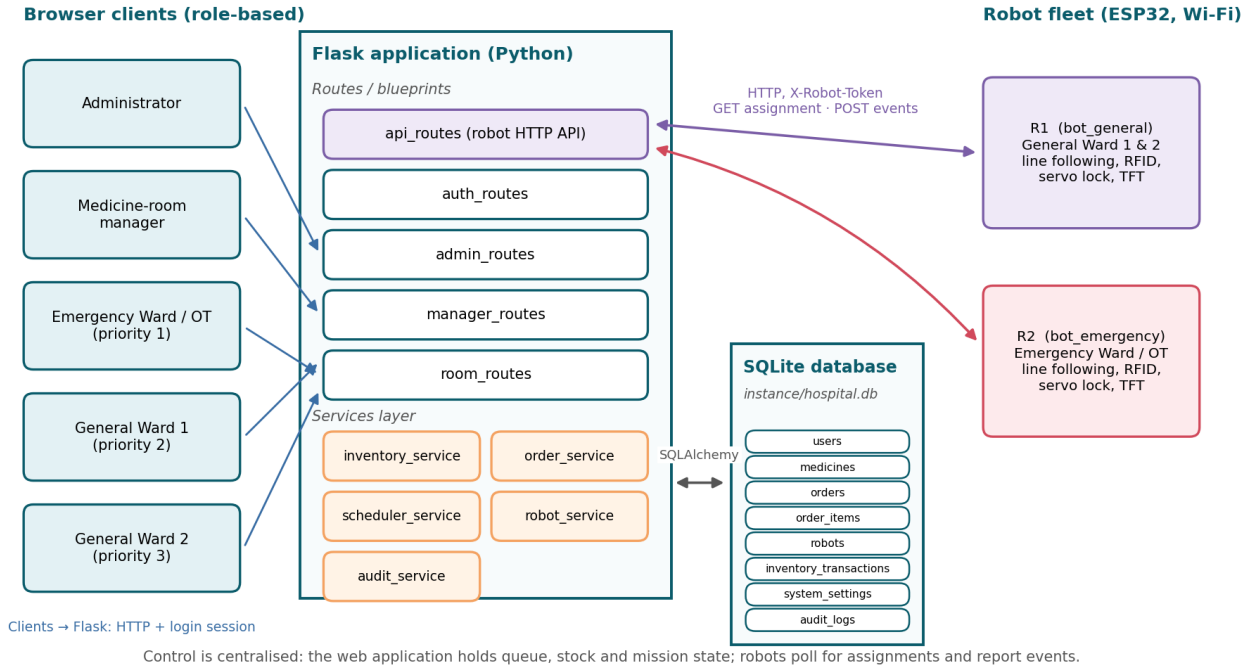


Figure 1. System architecture: role-based browsers, Flask application, SQLite database and the R1/R2 robot clients.

2.2 Software layers

Table 3. Source modules and responsibilities (from the review report)

Layer	Source files	Responsibility
Application	`app.py`, `config.py`	App factory, blueprint registration, extensions, SQLite default, environment configuration
Browser routes	`routes/auth_routes.py`, `admin_routes.py`, `manager_routes.py`, `room_routes.py`	Login, role views, manager actions, ward carts, JSON status endpoints
Robot interface	`routes/api_routes.py`	Token-checked GET assignment and POST heartbeat, node, arrival, handover and return
Business services	`services/inventory_service.py`, `order_service.py`, `scheduler_service.py`, `robot_service.py`, `audit_service.py`	Stock arithmetic, legal order transitions, sorted queue, robot assignment rules, audit writes
Persistence	`models.py`, `seed.py`	Eight SQLAlchemy tables and demonstration seed data
Presentation	`templates/`, `static/`	Jinja pages, stylesheet, `app.js`, `manager.js`
Tests	`tests/`	pytest modules for auth, inventory, scheduler, robot and audit behaviour

2.3 Data flow for one order

1. A ward user submits a cart. `create_order()` validates the items, reserves stock, stores the order with a snapshot of the ward priority and writes an audit entry.
2. The manager dashboard calls `get_active_orders_queue()` and shows orders sorted by priority, creation time and id.
3. The manager accepts the order, marks it prepared and assigns an available robot. `assign_robot_to_order()` links the order and the robot record.
4. The manager confirms loading and secures the box. The order becomes READY_FOR_DISPATCH.
5. The manager dispatches. In one transaction the order becomes DISPATCHED, physical stock is deducted, the reservation is released and an inventory transaction is written.
6. The robot firmware polls `GET /api/robot/<code>/assignment`, sees the order and starts its journey.
7. The robot reports arrival, handover and return through the API. The server updates the order and robot rows and writes audit entries.
8. On return confirmation the robot record is released and can be assigned again.

2.4 Communication model

- **Transport:** HTTP with JSON bodies. The server listens on `0.0.0.0:5000`, so a laptop on the same Wi-Fi network is reachable by IP address.
- **Authentication:** every robot request carries an `X-Robot-Token` header. The robot API is exempt from form CSRF protection and relies on this token.
- **Polling:** the firmware polls for assignments every 3 s (R1) or 2 s (R2), only while idle or waiting for loading.
- **Heartbeat:** the firmware posts status, a coarse location and a battery value every 5 s (R1) or 4 s (R2).
- **Not present:** there is no MQTT broker, no WebSocket or push channel, no timeout watchdog on the server side and no periodic polling in the browser JavaScript.

> Control is **centralised**. The web application holds the queue, the stock and the mission state, and a manager chooses the robot. This is not swarm intelligence, and the report is right to avoid describing it that way.

3. Users, Roles and Web Interface

3.1 Roles and permissions

The application has five roles. Access is enforced on the server by route decorators, and ward routes also compare the signed-in role with the requested ward code.

Table 4. Roles, priorities and landing pages

Role	Code	Priority	Main capabilities	Landing page
Emergency Ward / Operation Theatre	`emergency`	1 (highest)	Order medicines, see stock, track order and robot	`/ward/emergency`
General Ward 1	`general_1`	2	Order medicines, track status and assigned robot	`/ward/general_1`
General Ward 2	`general_2`	3	Order medicines, track status and assigned robot	`/ward/general_2`
Central Medicine Room manager	`medicine_manager`	Station control	Accept or reject, prepare, assign robot, confirm loading, dispatch, receive stock	`/manager`
Hospital administrator	`admin`	Superuser	Users, catalog, stock adjustment, robots, priority ranks, audit trail	`/admin/overview`

Landing pages follow the README and the route inspection in the review report. The setup guide uses `/room/<code>` for the ward pages, which appears to be an older path (see Chapter 16).

3.2 Screens

Table 5. Web interface screens and the actions behind them

Screen	Role	Purpose and key actions	Backend behaviour
Login	All	Username and password sign-in; shows demo accounts	Verifies password hash and active flag, starts a session, redirects by role, logs success or failure
Operations dashboard	Admin	Read-only counts: medicines, low stock, orders, R1/R2 status, recent audit events	Aggregates Medicine, Order, Robot and AuditLog rows
User and role management	Admin	Create accounts; activate, deactivate or reset passwords	Rejects duplicate usernames, hashes passwords, blocks self-deactivation, writes audit entries
Medicine catalog	Admin	Add and edit medicines (name, category, unit, stock, reorder threshold); activate or deactivate	Rejects duplicate names, clamps negative inputs to zero, audits changes
Order records	Admin	Filter by ward, status and priority; cancel before dispatch	Sorted newest first; cancellation releases reservations and unassigns a robot
Robot fleet	Admin	View R1 and R2; toggle active or maintenance	Refused while a robot has a current order
Priority configuration	Admin	Set the three ward ranks	Requires three distinct integers; stored as system settings; existing orders keep their stored priority
Audit trail	Admin	Last 200 events with actor, action, entity and details	Read-only; actions call `log_audit()`
Medicine-room dashboard	Manager	Priority queue, robot cards, counts; lifecycle actions on each order	Action route maps accept, reject, prepare, assign, load, secure, dispatch to service calls
Inventory and stock reception	Manager	Record inward stock with supplier or batch note	Positive quantity only; writes STOCK_IN transaction and audit entry

Screen	Role	Purpose and key actions	Backend behaviour
Ward requisition	Ward roles	Search and filter medicines, set quantities, add notes, place order	Validates active items and available stock, reserves stock, creates the order at the ward rank

3.3 Security measures in the application

- Passwords are stored as Werkzeug hashes and sessions are handled by Flask-Login.
- Role decorators protect administrator and manager routes; ward routes check the role against the ward code.
- Flask-WTF CSRF protection covers browser forms; the robot API uses a token instead.
- Login, user, stock, order, robot and settings actions are written to the audit log.

Before any real deployment: replace the demonstration accounts and development secret fallbacks, serve over HTTPS, rotate the robot token, review audit-log retention and run a security review. The review report states plainly that these measures are code paths, not a production security certification.

4. Inventory, Orders and Priority Scheduling

4.1 Stock model

Every medicine has a physical stock count **P**, a reserved quantity **R** and a reorder threshold. The quantity a ward can still order is the available stock:

> **Available stock A = Physical stock P − Reserved stock R** (the displayed value is clamped at zero).

Reservation and physical deduction are deliberately separate. Reserving stock commits units to an order the moment it is placed, so two wards cannot claim the same units. Physical stock only falls when the manager dispatches the order. Neither event proves that the medicine was actually put into a robot or received by a ward.

Table 6. Effect of each event on stock

Event	Physical P	Reserved R	Record written
Ward submits a valid order of quantity q	unchanged	R + q	Order and items; audit entry
Order rejected (PENDING only) or cancelled before dispatch	unchanged	R − q	Audit entry; any robot link cleared
Manager dispatches the order	P − q	R − q	Inventory transaction DISPATCH_DEDUCTION, −q
Manager records inward stock of n units	P + n	unchanged	Inventory transaction STOCK_IN, +n; audit entry
Administrator adjusts physical stock	set to new value	unchanged	Adjustment transaction; refused if below reserved units or negative

`reserve_stock_for_order()` rejects inactive or unknown medicines, non-positive quantities and requests above available stock. Repeated occurrences of the same medicine in a cart are added together before the check. A medicine is shown as LOW STOCK when 0 < A ≤ threshold.

Table 7. Illustrative arithmetic (not a measured result)

Step	P	R	A	Stock transaction
Starting state	20	4	16	–
Order for q = 3 reserved	20	7	13	–
Same order dispatched	17	4	13	−3 (DISPATCH_DEDUCTION)
Same order cancelled instead of dispatched	20	4	16	–

Available stock is the same immediately before and after dispatch (13 in the example), which is a useful invariant to check in tests.

4.2 Atomic dispatch and duplicate protection

When the manager confirms that the medicines are loaded and clicks the dispatch action, the following changes belong together: the order becomes DISPATCHED, physical stock is deducted, the reservation is released and an immutable record is added to `inventory_transactions`. A repeated request or a double-click is rejected with a clear error, so stock cannot be deducted twice. Backend tests H and I (exactly-once deduction and idempotency) cover this.

> The review report points out that the SQLite reservation path does not use a row lock, so behaviour under truly simultaneous submissions is not established. A concurrent-order test is still needed (scenario S4 and the concurrency item in Chapter 15).

4.3 Order lifecycle

The order status values below come from `services/order_service.py`. A robot is attached to an order while it is ACCEPTED, PREPARING or READY_FOR_LOADING, but "assigned" is not a separate order status.

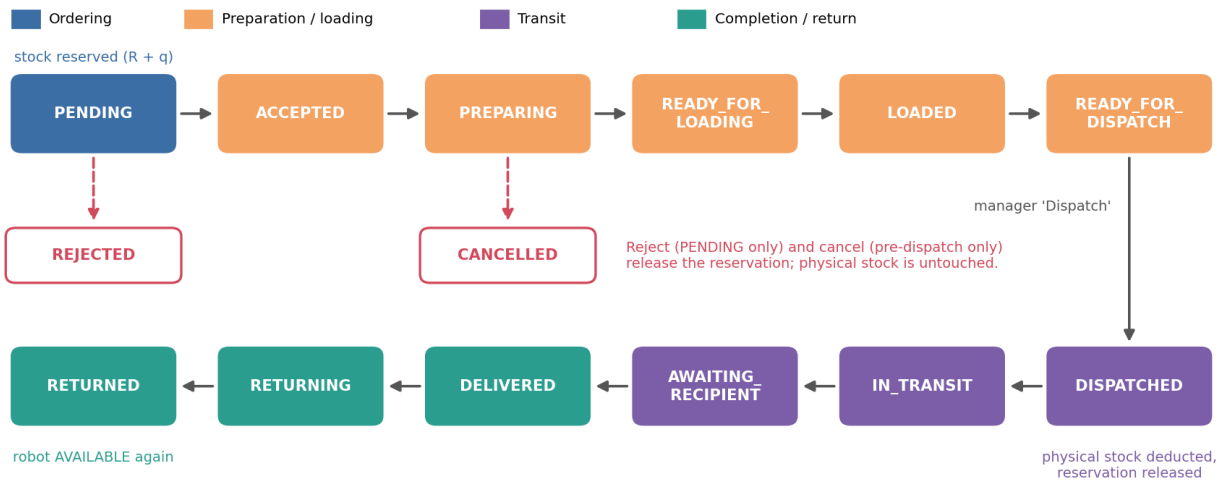


Figure 2. Order lifecycle and the stock effect at each stage.

Table 8. Order status definitions

Status	Meaning	Allowed progression or boundary
PENDING	Ward order stored; stock reserved	Manager may accept or reject; cancellation allowed
ACCEPTED / PREPARING	Accepted and under preparation	Manager marks PREPARING, then READY_FOR_LOADING; a robot may be assigned
READY_FOR_LOADING / LOADED	Prepared; then loading confirmed on the assigned robot	Securing the box changes LOADED to READY_FOR_DISPATCH
READY_FOR_DISPATCH	Box secured and dispatchable	Requires an assigned, active robot
DISPATCHED	Physical stock deducted in software	Repeated dispatch is rejected; cancellation no longer allowed
IN_TRANSIT / AWAITING_RECIPIENT	Reported travel; arrival at the ward	Advanced by manager action or token-checked robot event
DELIVERED / RETURNING / RETURNED	Handover reported; robot travelling back; return confirmed	Robot becomes AVAILABLE when return is confirmed
REJECTED / CANCELLED	Terminal pre-dispatch outcomes	Reservation released; no cancellation after dispatch

Other behaviour worth knowing: order numbers are built from a UTC date segment and a short random identifier; illegal transitions raise `OrderWorkflowError` and show as feedback; the arrival route can first move a DISPATCHED order through IN_TRANSIT; and there is no general FAILED_DELIVERY state.

4.4 Priority scheduling

Table 9. Configured ward hierarchy

Priority rank	Ward	Setting key
1 (most urgent)	Emergency Ward / Operation Theatre	`priority_emergency`
2	General Ward 1	`priority_general_1`

Priority rank	Ward	Setting key
3	General Ward 2	`priority_general_2`

The administrator can change the ranks, but the three values must stay distinct. Each new order stores a **snapshot** of its ward rank, so a later settings change does not re-rank existing orders. The queue is the set of active pre-dispatch orders (PENDING through READY_FOR_DISPATCH), sorted by the key (priority, created_at, id):

```
INPUT : eligible active orders
FOR each order o:  key(o) = (o.priority, o.created_at, o.id)
SORT orders by ascending key
RETURN ordered list, positions 1..n

Dispatch suggestion: among READY_FOR_DISPATCH orders that have a robot,
                     return the order with the minimum key
```

This corresponds to `get_active_orders_queue()` and `get_next_dispatchable_order()`; `get_next_order_for_robot_assignment()` applies the same key to unassigned orders. The sort costs O(n log n) in time and O(n) in list storage per queue refresh, excluding database access.

Table 10. What "priority" does and does not mean in the current system

Behaviour	Status	Explanation
A new emergency order appears above waiting routine orders	Implemented	Deterministic queue key: rank, then FIFO, then id
The manager is forced to dispatch the first queue item	Not implemented	The action route accepts any valid order id; the queue is a decision aid
A queued, not-yet-dispatched assignment is automatically revoked and reassigned	Not implemented	Only manual cancellation or reassignment before dispatch
An active mission is interrupted to serve an urgent request	Future work	Needs a safe stop, payload custody, a new route and a resume or reassign rule
Aging or a maximum-wait rule to prevent starvation of lower ranks	Proposed	A steady stream of rank-1 orders can delay ranks 2 and 3 indefinitely

Design observation. Dedicating R2 to the Emergency Ward / OT means an emergency order does not normally have to wait for a general-ward delivery to finish. It does not remove the problem entirely: if R2 is already busy with an earlier emergency order, the next one waits, and nothing in the software currently stops a manager assigning R1 to an emergency order (see Chapter 7, observation F10).

5. Robot Fleet Management (Backend)

5.1 Robot records and states

`seed.py` creates two robot rows, **R1** (primary) and **R2** (secondary). A robot row holds its code, active flag, status, last reported location, battery percentage, last-seen time and current order. The README defines nine robot states:

Table 11. Robot states

State	Meaning
`AVAILABLE`	Docked in the medicine room and ready for assignment
`ASSIGNED`	Assigned to an accepted order
`LOADING`	Medicines confirmed as placed in the cargo box
`READY`	Cargo box secured and latched
`DISPATCHED`	Order completed on the server side; robot has left the station
`DELIVERING`	En route along the line track
`AWAITING_RECIPIENT`	Arrived at the ward, waiting for the recipient RFID unlock
`RETURNING`	Travelling back to the medicine room
`MAINTENANCE`	Offline for mechanical or battery servicing

5.2 Assignment rules

- `get_available_robots()` returns robots that are active, have status AVAILABLE and have no current order.
- The **manager chooses** the robot. `assign_robot_to_order()` checks those conditions and links `Order.robot_id` and `Robot.current_order_id`. It can replace an earlier robot association before dispatch.
- **Safety rule:** a robot cannot be assigned to another order until its return to the medicine room has been confirmed. Return confirmation sets the robot to AVAILABLE, clears its current order and updates location and last-seen time.
- The administrator can deactivate a robot or put it in maintenance only when no order is linked to it.
- There is no automatic first-available, nearest-robot, least-workload or battery-aware selection, and a low battery value does not block assignment.

5.3 Telemetry

The heartbeat stores the reported status, location and battery (clamped to 0 to 100) together with a last-seen timestamp. These are stored fields, not calibrated measurements: the current firmware sends a fixed battery constant (see Chapter 7, observation F6), and the location is a coarse label such as the medicine room or the destination ward name. No telemetry history table exists, so only the latest values are kept.

6. Robot Hardware

6.1 Overview

According to the setup guide, both robots use the same tested hardware and pin configuration. Each robot is an ESP32 Dev Module that reads two IR sensors for line following, drives two DC motors through an L298N H-bridge, locks the cargo lid with a servo, reads RFID cards with an RC522 and shows its state on an ILI9341 TFT. The only differences between R1 and R2 are in firmware (Chapter 7).

6.2 Components

> The files name the component types and pins but not exact models, ratings, motors, battery or chassis. The specification column below gives **typical datasheet values for each part class** as a starting point. Confirm each value against the modules actually purchased before quoting it in a report.

Table 12. Components and typical specifications

Component	Role	Interface (from the files)	Typical specification (verify)
ESP32 Dev Module	Controller and Wi-Fi client	USB 5 V or VIN; 3.3 V GPIO	Dual-core Xtensa LX6 up to 240 MHz, 520 KB SRAM, usually 4 MB flash, 802.11 b/g/n on 2.4 GHz only; GPIO not 5 V tolerant
2-channel IR line sensor	Line following and cross-line detection	Digital inputs: left GPIO 13, right GPIO 27	Reflective IR modules (often TCRT5000-type), 3.3 to 5 V, digital output with sensitivity trimmer, useful range from a few mm to about 2 cm
L298N motor driver	Drives left and right motors	ENA GPIO 3, ENB GPIO 1 (PWM), IN1 to IN4 on GPIO 17, 16, 14, 12	Dual H-bridge, motor supply about 5 to 35 V, about 2 A per channel, roughly 1.4 to 2 V drop per bridge
DC gear motors (2)	Drive wheels	Via L298N	Not documented. To record: voltage, gear ratio, no-load speed, stall current
Servo lock	Locks and unlocks the cargo lid	GPIO 15, 50 Hz, pulses 500 to 2400 µs; 45° = locked, 90° = open	Typical SG90 or MG90S class: 4.8 to 6 V supply, about 180° travel. Model not documented
RC522 RFID reader	Card-based access control	SPI: SS GPIO 5, RST GPIO 26, SCK 18, MISO 19, MOSI 23	13.56 MHz, ISO/IEC 14443A cards and tags, 3.3 V supply only, read range about 3 to 5 cm
ILI9341 TFT	Status display	SPI: CS GPIO 21, DC GPIO 22, RST GPIO 2, shared SCK/MISO/MOSI	2.4 inch or 2.8 inch, 240 × 320 pixels, 3.3 V logic; firmware uses landscape rotation 1
Battery and power distribution	Powers motors and logic	Not documented	To record: chemistry, voltage, capacity, regulator for ESP32 and servo, charging arrangement
Chassis, wheels, cargo box	Mechanical platform	Not documented	To record: dimensions, wheel diameter, payload, track width

6.3 Pin map

Table 13. GPIO assignments (identical on both robots)

Signal	GPIO	Peripheral	Note
Left IR sensor	13	IR line sensor	Digital input; LOW = line detected (`LINE_DETECTED_VAL = 0`)
Right IR sensor	27	IR line sensor	Digital input
Servo signal	15	Cargo lock servo	GPIO 15 is a boot strapping pin
RFID SS (SDA)	5	RC522	Chip select

Signal	GPIO	Peripheral	Note
RFID RST	26	RC522	
SPI SCK	18	RC522 and TFT	Shared bus
SPI MISO	19	RC522 and TFT	Shared bus
SPI MOSI	23	RC522 and TFT	Shared bus
TFT CS	21	ILI9341	Chip select
TFT DC	22	ILI9341	
TFT RST	2	ILI9341	GPIO 2 is a boot strapping pin
ENA (left speed)	3	L298N	This is UART0 RX; see observation F4
ENB (right speed)	1	L298N	This is UART0 TX; see observation F4
IN1	17	L298N	Left motor direction
IN2	16	L298N	Left motor direction
IN3	14	L298N	Right motor direction
IN4	12	L298N	GPIO 12 is a boot strapping pin; see observation F5

6.4 Wiring and power notes

These notes come from engineering practice for the listed parts and the pin map. They are recommendations, not records of what was tested on the robots.

- **RC522 on 3.3 V only.** The setup guide already warns against 5 V. The ESP32 GPIOs are not 5 V tolerant either.
- **Separate or well-decoupled motor supply.** Motor noise can reset the ESP32 or cause Wi-Fi drop-outs. Use a common ground, bulk and ceramic capacitors near the L298N and the servo, and a regulated 5 V rail for the servo rather than the ESP32 3.3 V pin.
- **Servo current spikes** at start and stall can brown out the controller if they share a weak regulator.
- **Free pins.** GPIO 4, 25, 32 and 33 are not used in this pin map and can drive outputs. GPIO 34 to 39 are input-only.
- **Shared SPI bus.** The firmware pulls one chip select high before using the other device. Keep SPI wires short to limit noise on the shared lines.

6.5 Build record to complete

The review report left a verification table for the physical build. It is reproduced here so the team can fill it in from the real robots, photographs and datasheets. A blank cell means "not yet recorded", not "absent".

Table 14. Physical build and interface verification record

Item	Actual part, rating or pin	Evidence
Number of robots built; chassis dimensions		
Motors, wheel geometry, drive voltage		
Battery, protection, charging, emergency stop		
IR sensor model and mounting height		
Servo model and lid mechanism		

Item	Actual part, rating or pin	Evidence
RFID cards issued (manager, each ward)		
Track: dimensions, line width, marker positions		
Measured speed, turn timings, battery runtime		

7. Robot Firmware

7.1 The two sketches

The firmware is written for the ESP32 Dev Module in the Arduino framework. `bot_general.ino` runs **R1** and handles General Ward 1 and General Ward 2. `bot_emergency.ino` runs **R2** and handles the Emergency Ward / OT. About two thirds of the code is shared: motor control, servo lock, RFID reading, TFT drawing and the HTTP client. The sketches differ in navigation logic, display theme, timing and card rules.

Table 15. Differences between the two sketches

Item	bot_general.ino (R1)	bot_emergency.ino (R2)
`ROBOT_CODE`	"R1"	"R2"
Destinations	General Ward 1, General Ward 2	Emergency Ward / OT
Priority on display	From the order, colour-coded (1 red, 2 green, 3 orange)	Fixed text "PRIORITY 1 (CRITICAL)"
Display theme	Navy and cyan	Red
Assignment poll interval	3000 ms	2000 ms
Heartbeat interval	5000 ms	4000 ms
`MOTOR_SPEED` / `TURN_SPEED` (PWM 0 to 255)	190 / 150	210 / 165
Navigation phases	Eight-value `NavPhase` enum	None: one outbound and one return segment
Decision point	Yes: straight for Ward 1, 90° right for Ward 2	None
90° turn timing	600 ms	Not used
180° turn / cross-line clearing	1200 ms / 400 ms	1200 ms / 400 ms
Pause on reaching a marker	200 ms	500 ms
Return behaviour	Ward 1: pass straight through the decision point. Ward 2: 90° left at the junction	Any cross-line while returning is treated as the dock
Cards accepted	Universal manager card, Ward 1 card, Ward 2 card	Universal manager card, Emergency card
Card constants in the file	All three set to the same UID	Three different placeholder UIDs
Server URL constant	Written without `http://`	Includes `http://`
Reported battery constant	95	98
Medicine text shown on TFT	First 48 characters	First 45 characters

7.2 Program structure

`setup()` initialises, in order: serial at 115200 baud; motor direction pins and two LEDC PWM channels (1 kHz, 8-bit, with code paths for both Arduino-ESP32 core 2.x and 3.x); the IR input pins; the servo (50 Hz, 500 to 2400 µs pulses) which is immediately locked; the chip-select pins; the SPI bus; the TFT in landscape rotation; the RFID reader; and finally Wi-Fi, with up to 20 connection attempts of 500 ms each. If Wi-Fi does not connect, the robot continues in "standalone mode" and the display shows WIFI: OFF.

`loop()` repeats the same sequence every 20 ms:

1. Read the RFID reader and, if an authorised card is present, act on it according to the current state.
2. If the state is IN_TRANSIT or RETURNING, run `handleLineTracking()`.

3. If the state is IDLE or ORDER_ASSIGNED and the poll interval has elapsed, call `pollServerAssignment()`.
4. If the heartbeat interval has elapsed, call `sendHeartbeat()` (in every state).

Table 16. Main functions

Function	Purpose
`setMotorSpeed`, `stopMotors`, `moveForward`, `turnLeft`, `turnRight`	Drive the L298N direction pins and set PWM speed on ENA and ENB
`turn180Degrees`, `turn90DegreesRight`, `turn90DegreesLeft` (R1 only)	Open-loop pivots: stop, pivot for a fixed time, stop
`lockBox`, `unlockBox`, `toggleBox`	Move the servo to 45° (locked) or 90° (open) and track `boxIsOpen`
`updateTFTDisplay`, `showDecisionPointUI` (R1 only)	Redraw the full status screen; show the decision-point banner
`pollServerAssignment`	GET the assignment, parse JSON, update `currentOrder`, start the mission when ready
`notifyArrivalAtWard`, `notifyBoxClosedAndDelivered`, `notifyReturnedToBase`	POST the arrival, handover and return events
`sendHeartbeat`	POST status, location and battery
`readRFIDCardUID`, `isAuthorizedCard`	Read a card as "XX XX XX XX" uppercase hex and check it against the allowed UIDs
`handleLineTracking`	Two-sensor line following and marker handling

7.3 State machine

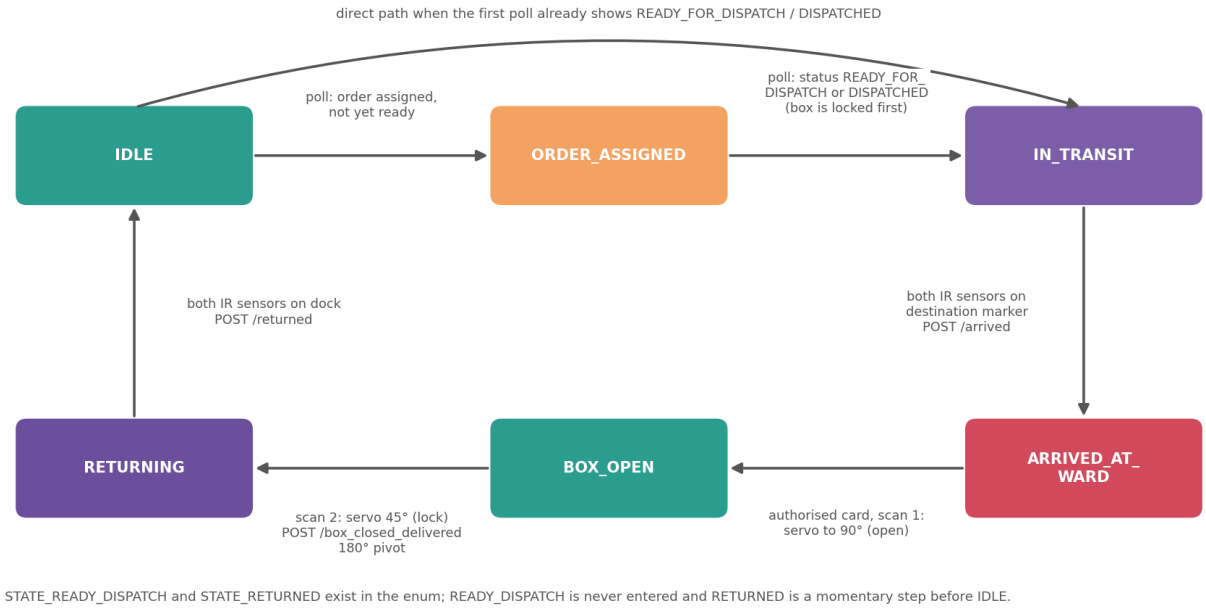


Figure 3. Firmware state machine (both sketches; R1 adds navigation sub-phases, shown in Figure 4).

Table 17. Firmware states and display banners

State	Entered when	Banner on TFT (R1 / R2)
`STATE_IDLE`	Power-up, and after docking	IDLE - READY AT BASE / READY AT BASE - STANDBY
`STATE_ORDER_ASSIGNED`	A poll returns an order that is not yet ready for dispatch	ORDER ASSIGNED (LOADING) / EMERGENCY ORDER ASSIGNED

State	Entered when	Banner on TFT (R1 / R2)
`STATE_IN_TRANSIT`	A poll returns status READY_FOR_DISPATCH or DISPATCHED	EN ROUTE TO WARD / PRIORITY 1: EN ROUTE TO OT
`STATE_ARRIVED_AT_WARD`	Both IR sensors see the destination marker	ARRIVED - SCAN RFID CARD / ARRIVED OT - SCAN RFID
`STATE_BOX_OPEN`	First authorised card scan at the ward	BOX UNLOCKED - RETRIEVE
`STATE_RETURNING`	Second authorised card scan at the ward	RETURNING TO PHARMACY
`STATE_RETURNED`	Momentary, set just before returning to IDLE	MISSION COMPLETED
`STATE_READY_DISPATCH`	Declared and given a banner, but never entered by the code	READY FOR DISPATCH / READY FOR EMERGENCY DISPATCH

7.4 Navigation

Line following

Each IR sensor reports a digital value, and `LINE_DETECTED_VAL = 0` means a LOW output counts as "on the line". The comments in the code ("neither sensor directly on edge, move straight") indicate that the two sensors straddle the black line, so seeing no line on either side means the robot is centred. The steering rule is simple on/off (bang-bang) control, not proportional control:

Table 18. Steering rule in `handleLineTracking()`

Left sensor	Right sensor	Meaning	Action
on line	on line	Cross-line marker (intersection, ward or dock)	Stop, then handle the marker (below)
on line	off line	Line drifting to the left	`turnLeft()` (left motor reverse, right forward)
off line	on line	Line drifting to the right	`turnRight()` (left motor forward, right reverse)
off line	off line	Centred on the line	`moveForward()`

Track layout and marker handling (R1)

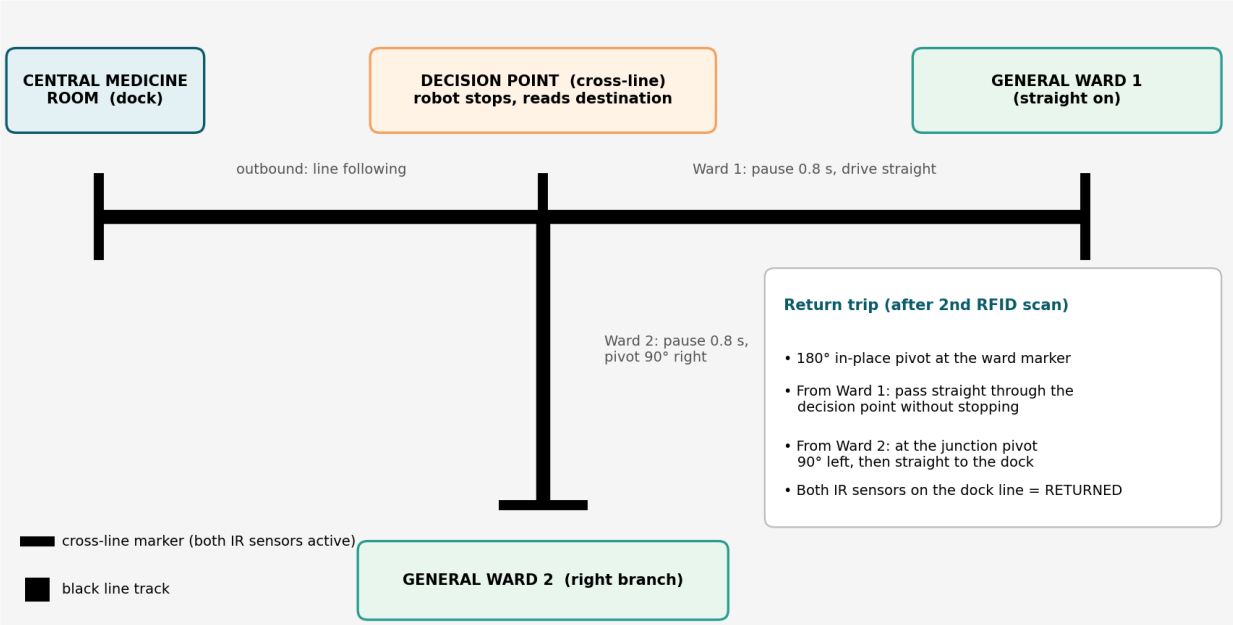


Figure 4. Track layout implied by the R1 firmware and the setup guide (not to scale). The R2 route is not documented in the supplied files.

Table 19. How R1 reacts to a cross-line marker

Case	State and phase	Action
A	IN_TRANSIT, `NAV_OUTBOUND_TO_DECISION`	Stop at the decision point and show the banner. If the destination text contains "Ward 1": wait 0.8 s, drive 400 ms to clear the line, phase becomes `NAV_OUTBOUND_TO_WARD1`. Otherwise: wait 0.8 s, pivot 90° right (600 ms), phase becomes `NAV_OUTBOUND_TO_WARD2`
B	IN_TRANSIT, `NAV_OUTBOUND_TO_WARD1` or `_WARD2`	Destination marker reached: state ARRIVED_AT_WARD, POST arrival
C	RETURNING, `NAV_RETURN_FROM_WARD1`	Decision point crossed without stopping; phase becomes `NAV_RETURN_TO_BASE`
D	RETURNING, `NAV_RETURN_FROM_WARD2`	Pivot 90° left (600 ms) towards the medicine room; phase becomes `NAV_RETURN_TO_BASE`
E	RETURNING, `NAV_RETURN_TO_BASE`	Dock reached: POST returned, clear the order, state IDLE

R2 has no decision point. While IN_TRANSIT, the first cross-line it sees is the Emergency Ward / OT marker; while RETURNING, the first cross-line it sees is the dock.

> The 90° and 180° turns are **timed** (600 ms and 1200 ms at a fixed PWM), not measured. The angle actually turned depends on battery voltage, floor friction and payload, so these constants must be recalibrated for each robot and battery state.

7.5 RFID access control

A card is read as its UID in the form "XX XX XX XX". The `isAuthorizedCard()` function accepts the **universal manager card** in every situation, and a ward card only when the current destination matches (R1 checks the destination text for "Ward 1" or "Ward 2"; R2 accepts the emergency card). What an accepted card does depends on the state:

Table 20. Card behaviour by state

State	Scan	Effect
IDLE or ORDER_ASSIGNED (at the medicine room)	Universal card	Toggles the lid: closed becomes open (90°), open becomes closed (45°). Two scans bracket the loading step
ARRIVED_AT_WARD	Universal or matching ward card	Unlocks the lid (90°), state BOX_OPEN
BOX_OPEN	Universal or matching ward card	Locks the lid (45°), POST handover, wait 0.5 s, 180° pivot, step forward 400 ms, state RETURNING
IN_TRANSIT or RETURNING	Any	Ignored

The reader is polled on every loop iteration. After a successful read the card is halted and crypto is stopped so the next tap is seen as a new card.

7.6 Server communication

Table 21. HTTP calls made by the firmware

Function	Request	When	Body and notes
`pollServerAssignment`	`GET /api/robot/<code>/assignment`	Every 3 s (R1) or 2 s (R2) while IDLE or ORDER_ASSIGNED	Expects `has_assignment` and an `order` object with `order_id`, `order_number`, `destination_room`, `priority`, `status` and `items[]` (`medicine`, `quantity`). Parsed into a 1024-byte JSON buffer
`notifyArrivalAtWard`	`POST .../arrived`	On reaching the destination marker	Empty JSON object
`notifyBoxClosedAndDelivered`	`POST .../box_closed_delivered`	On the second ward card scan	Empty JSON object
`notifyReturnedToBase`	`POST .../returned`	On reaching the dock marker	Empty JSON object
`sendHeartbeat`	`POST .../heartbeat`	Every 5 s (R1) or 4 s (R2), in every state	`{"status", "location", "battery"}`

All requests add the `X-Robot-Token` header. The firmware never calls `POST .../rfid_node`.

Table 22. How the firmware chooses the heartbeat fields

Firmware state	`status` sent	`location` sent
IN_TRANSIT	DELIVERING	Destination ward name
ARRIVED_AT_WARD	AWAITING_RECIPIENT	Destination ward name
RETURNING	RETURNING	Destination ward name
IDLE, ORDER_ASSIGNED, BOX_OPEN and others	AVAILABLE	"Medicine Room" when IDLE, otherwise the destination name

A mission starts when a poll returns an order with status READY_FOR_DISPATCH or DISPATCHED while the robot is IDLE or ORDER_ASSIGNED. The firmware then locks the lid, sets IN_TRANSIT, redraws the display and drives forward 400 ms to leave the docking line.

7.7 Display

The 320 × 240 landscape screen shows a header bar with the robot name and WIFI: OK or WIFI: OFF, a colour-coded state banner, the order number with a BOX: OPEN or BOX: CLOSED badge, the destination, the priority,

the first characters of the medicine list and a footer hint that depends on the state. The TFT and RFID reader share the SPI bus, so the code raises one chip select and lowers the other around each use.

7.8 Tunable parameters

Table 23. Constants to calibrate on the real robots

Constant	R1 / R2 value	Effect and tuning note
`MOTOR_SPEED`	190 / 210	Forward PWM (0 to 255). Lower it if the robot overshoots curves
`TURN_SPEED`	150 / 165	PWM used for steering and pivots
`PWM_FREQ`, `PWM_RESOLUTION`	1000 Hz, 8 bit	LEDC settings for ENA and ENB
`TURN_180_TIME_MS`	1200	Duration of the 180° pivot. Recalibrate per robot and battery level
`TURN_90_TIME_MS`	600 (R1 only)	Duration of the 90° pivot
`CROSSING_CLEAR_MS`	400	Forward drive to clear a cross-line so it is not detected twice
`POLL_INTERVAL`	3000 / 2000 ms	Assignment polling rate
`HEARTBEAT_INTERVAL`	5000 / 4000 ms	Telemetry rate
`SERVO_LOCKED_ANGLE`, `SERVO_UNLOCKED_ANGLE`	45°, 90°	Lid positions; adjust to the mechanism
`LINE_DETECTED_VAL`	0	Set to 1 for sensor modules with active-high output
Main loop delay	20 ms	Sets how often sensors and the card reader are sampled
Wi-Fi attempts	20 × 500 ms	About 10 s before standalone mode

7.9 Code review observations

The observations below come from reading the two sketches against the setup guide, the README and the review report. They are not the result of running the robots, so each should be confirmed on hardware. Items marked High are the ones most likely to stop a demonstration from working or to cause unsafe behaviour.

Table 24. Firmware observations and suggested actions

ID	Priority	Observation	Suggested action
F1	High	`SERVER_BASE_URL` in `bot_general.ino` has no `http://` prefix (R2 has it). HTTPClient needs a full URL, so R1's requests are expected to fail while the display still shows WIFI: OK.	Use `http://<laptop-ip>:5000` and print the result of `http.begin()`
F2	High	Both sketches hold a Wi-Fi name and password and the robot API token in plain text (values not reproduced here).	Remove before sharing, uploading or submitting. Move to a header file excluded from version control. Rotate the password and token
F3	High	The firmware leaves the dock on status READY_FOR_DISPATCH or DISPATCHED. The setup guide says the robot starts on DISPATCHED. If the assignment route returns the order while it is READY_FOR_DISPATCH, the robot leaves before the manager's dispatch click deducts stock.	Decide the intended gate. If dispatch must authorise departure, start only on DISPATCHED
F4	High	ENA and ENB use GPIO 3 and GPIO 1, which are UART0 RX and TX, while `Serial.begin(115200)` also uses them. Serial output and motor PWM can interfere, and boot messages can pulse ENB.	Move ENA and ENB to free pins (for example GPIO 25 and 32) or avoid serial use in the field build. Confirm on the real boards
F5	Medium	IN4 uses GPIO 12, a strapping pin that selects flash voltage. If the L298N input holds it high at reset, boot can fail. GPIO 2 (TFT reset) and GPIO 15 (servo) are also strapping pins.	Move IN4 to a free pin, or make sure it is low or floating at reset

ID	Priority	Observation	Suggested action
F6	Medium	`batteryPercentage` is a constant (95 on R1, 98 on R2) and is never measured, yet the server stores it as telemetry.	Add a voltage divider and ADC reading with a low-battery cut-off, or label the value as simulated
F7	Medium	In `bot_general.ino` the universal, Ward 1 and Ward 2 card constants are identical, so one card works for every role on R1. R2 uses three different placeholder UIDs.	Read each real card, store distinct UIDs and keep the manager card separate from ward cards
F8	Low	The R1 header comment describes an "8-channel IR sensor" and "proportional" tracking, but the code reads two digital sensors with on/off steering (as the setup guide says).	Correct the comment so code, guide and report agree
F9	Medium	The heartbeat POST runs in every state, including IN_TRANSIT and RETURNING, and blocks until a reply or timeout. The motors keep their last command meanwhile, so the robot may overshoot a turn or leave the line. Manoeuvres also use blocking `delay()`.	Send heartbeats only when stopped, shorten `http.setTimeout`, or run networking in a separate FreeRTOS task
F10	High	R1 treats any destination without "Ward 1" as Ward 2 and turns right at the decision point; R2 has no destination check. The backend lets a manager assign any available robot to any order, so R1 given an emergency order would drive to the Ward 2 branch.	Enforce R1 for general wards and R2 for emergency in `assign_robot_to_order()` and add a destination guard in the firmware
F11	Medium	Arrival is detected by both IR sensors on a cross-line, not by RFID, and the firmware never calls `/rfid_node`. The README wording ("detects destination RFID tag") does not match. R2 treats its first cross-line as the destination, so any extra line on its track gives a false arrival.	Align the README and report with the code. Keep R2's track free of other cross-lines, or add route phases
F12	Medium	Turns are timed and open-loop (600 ms and 1200 ms at fixed PWM) and depend on battery voltage, floor and payload.	Recalibrate per robot. Consider wheel encoders or an IMU as a proposed upgrade
F13	Medium	Arrival, handover and return POST results are printed but not checked, and there is no Wi-Fi reconnect code after `setup()`. A drop at the wrong moment leaves the server unaware of an event the robot has already acted on.	Retry each event until HTTP 200 and call `WiFi.reconnect()` when the link is lost
F14	Medium	If an order is cancelled or unassigned while the robot is in ORDER_ASSIGNED, a poll with `has_assignment = false` only prints a message. The old order stays on the display and the state does not return to IDLE.	Reset `currentOrder` and the state to IDLE when no assignment is returned
F15	Low	The heartbeat sends AVAILABLE in ORDER_ASSIGNED and BOX_OPEN. Depending on how the server handles telemetry status, a loaded robot could be shown as AVAILABLE.	Check `update_robot_telemetry()` and map the remaining states explicitly
F16	Low	The assignment reply is parsed into a 1024-byte `StaticJsonDocument`. A long manifest may not fit. The TFT shows only 45 to 48 characters of it.	Size the buffer for the largest order, or filter the JSON. Add a scrolling or paged manifest view

> Not present in either sketch (consistent with the review report): obstacle sensing, an emergency stop, a low-battery interlock, a timeout if nobody scans the ward card, and any route planning. These are future work, not defects in the existing code.

8. Robot REST API

The robot API lives in `routes/api_routes.py`. Every request must carry the `X-Robot-Token` header with the token configured on the server. The token value is not reproduced in this document.

Table 25. Robot API endpoints

Method and path	Purpose	Used by firmware	Payload
`GET /api/robot/<code>/assignment`	Current assignment, destination, priority and medicine manifest	Yes (polling)	Reply described below
`POST /api/robot/<code>/heartbeat`	Status, location and battery telemetry; battery clamped to 0 to 100	Yes	`{"status": "DELIVERING", "location": "Corridor A", "battery": 92}` (README example)
`POST /api/robot/<code>/rfid_node`	Waypoint checkpoint report	No	`{"node_id": "TAG-8821", "node_label": "Junction 3"}` (README example)
`POST /api/robot/<code>/arrived`	Destination reached; notifies ward staff. May first advance a DISPATCHED order through IN_TRANSIT	Yes	Empty object
`POST /api/robot/<code>/box_closed_delivered`	Box unlocked, medicine removed, lid latched	Yes	Empty object
`POST /api/robot/<code>/returned`	Docked at the medicine room; robot becomes AVAILABLE	Yes	Empty object

The assignment reply has the shape below. It is reconstructed from what the firmware parses; check it against the real route before relying on field names or types.

```
{
  "has_assignment": true,
  "order": {
    "order_id": 17,
    "order_number": "<generated number>",
    "destination_room": "General Ward 1",
    "priority": 2,
    "status": "READY_FOR_DISPATCH",
    "items": [ { "medicine": "<name>", "quantity": 3 } ]
  }
}
```

A quick test of the interface from a laptop (replace the placeholders):

```
curl -H "X-Robot-Token: <token>" http://<laptop-ip>:5000/api/robot/R1/assignment
```

A positive HTTP reply is a software event only. It is not proof that the robot physically arrived, that the recipient was identified or that the medicine was handed over.

9. End-to-End Operating Workflow

The table follows one order from request to return, combining the web application behaviour with what the firmware does at each step.

Table 26. Order workflow across web application and robot

#	Actor	Web application	Robot and firmware	Result
1	Ward nurse	Adds medicines to the cart and places the order	Idle, polling every few seconds	Order PENDING; stock reserved
2	Manager	Accepts the order from the priority queue	–	ACCEPTED
3	Manager	Prepares the order and assigns R1 (general wards) or R2 (emergency)	Poll returns the order; state ORDER_ASSIGNED; TFT shows order id, destination and medicines	PREPARING, then READY_FOR_LOADING; robot linked
4	Manager	Taps the universal manager card on the robot, loads the medicines, taps again, then ticks Confirm Medicines Loaded	Scan 1: lid opens (90°), BOX: OPEN. Scan 2: lid locks (45°), BOX: CLOSED	LOADED, then READY_FOR_DISPATCH
5	Manager	Clicks Dispatch Robot	Sees READY_FOR_DISPATCH or DISPATCHED, locks the lid, leaves the dock	DISPATCHED; physical stock deducted; reservation released
6	Robot	Transit is reported	Follows the line. R1 stops at the decision point: straight for Ward 1, 90° right for Ward 2	IN_TRANSIT
7	Robot	Arrival recorded	Stops at the ward marker; POST arrived; TFT asks for an RFID card	AWAITING_RECIPIENT; ward staff notified
8	Ward nurse	Handover recorded	Scan 1 unlocks the lid; nurse removes the medicine; scan 2 locks it; POST handover; 180° pivot	DELIVERED
9	Robot	Return reported	R1 from Ward 1 passes straight through the decision point; from Ward 2 turns 90° left. R2 drives back	RETURNING
10	Robot	Return confirmed	Stops at the dock marker; POST returned; state IDLE	RETURNED; robot AVAILABLE for the next order

The setup guide describes the manager step as "Dispatch Robot" with RFID scans, while the README calls the button "Complete & Dispatch" and does not mention the scans. Both describe the same dispatch action; the scans are done on the robot, not in the web application.

10. Database Design

`models.py` defines eight SQLAlchemy tables. The default database is the SQLite file `instance/hospital.db`; `app.py` calls `db.create_all()` and `seed.py` fills it with demonstration data. No migration history is supplied.

Table 27. Tables, keys and relationships (from the review report)

Table	Primary key and important fields	Foreign keys and relationships
`users`	id; unique username, password_hash, role, room_name, active	One user creates many orders and can appear in audit and stock records
`medicines`	id; unique name, category, unit, physical_stock, reserved_quantity, reorder_threshold, active	Referenced by order_items and inventory_transactions
`orders`	id; unique order_number, room_name, priority (snapshot), status, milestone timestamps	`user_id` to users; `robot_id` to robots (nullable)
`order_items`	id; quantity, unit	`order_id` to orders; `medicine_id` to medicines
`robots`	id; unique robot_code, status, active, last_location, last_seen, battery_percentage	`current_order_id` to orders (nullable), separate from `orders.robot_id`
`inventory_transactions`	id; type, signed quantity_change, reason, created_at	`medicine_id`, `user_id`; `order_id` nullable
`system_settings`	id; unique setting_key, setting_value	Stores the three ward ranks; no foreign keys
`audit_logs`	id; action, entity_type, entity_id, details, created_at	`user_id` to users (nullable for system events)

- Role and room are **text fields** on users and orders, not separate tables.
- A mission is represented by order and robot statuses and timestamps. There is **no** mission, route or telemetry-history table.
- The two links `orders.robot_id` and `robots.current_order_id` are independent and must be kept in step by the service functions.
- Available stock is computed, not stored. Services enforce workflow rules beyond what foreign keys can express.
- Order creation reserves all items in one session and commits the order; dispatch changes both stock counts and writes transaction rows. Audit writes may use separate commits, so their atomicity with business actions should be tested.

11. Installation, Configuration and Operation

11.1 Prerequisites

- Python 3.10, 3.11 or 3.12, installed with "Add Python to PATH" ticked.
- Arduino IDE 2.0 or newer.
- A CP2102 or CH340 USB driver if the ESP32 COM port is not detected (Windows 10 and 11 usually detect it automatically).
- A 2.4 GHz Wi-Fi network or phone hotspot shared by the laptop and the robots. The ESP32 does not support 5 GHz.

11.2 Backend setup (Windows PowerShell)

```
cd "C:\path\to\hospital medicine transport bot"
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest -v                # all tests should pass
python seed.py           # creates instance/hospital.db with demo data
python app.py            # serves on http://0.0.0.0:5000
```

If PowerShell blocks the activation script, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` and activate again. In Command Prompt use `.venv\Scripts\activate.bat`. Keep the terminal open while the system is in use, then browse to `http://127.0.0.1:5000` on the laptop or `http://<laptop-ip>:5000` from another device.

11.3 Network

- Run `ipconfig` and note the IPv4 address under the Wi-Fi adapter (for example 192.168.1.10).
- Test from a phone browser on the same network. If the page opens, the robots can reach the server too.
- Allow `python.exe` through Windows Defender Firewall for both private and public networks.

11.4 Arduino IDE setup

1. Open File, Preferences and add `https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json` to Additional Boards Manager URLs.
2. In Boards Manager install "esp32" by Espressif Systems.
3. In Library Manager install: Adafruit ILI9341, Adafruit GFX Library, MFRC522, ESP32Servo (Kevin Harrington) and ArduinoJson (6.x or 7.x).

11.5 Configuring and uploading the firmware

Near the top of each sketch, edit these constants before uploading:

Constant	What to set
`WIFI_SSID`, `WIFI_PASSWORD`	Your 2.4 GHz network or hotspot
`SERVER_BASE_URL`	`http://<laptop-ip>:5000`, **including `http://`** (see observation F1) and never localhost
`ROBOT_TOKEN`	Must match the token configured on the server
`ROBOT_CODE`	"R1" in `bot_general.ino`, "R2" in `bot_emergency.ino`

Constant	What to set
`CARD_*` UIDs	The real UIDs of the manager card and ward cards (see observation F7)
Timing constants	Calibrate turn and crossing times on the real track (Section 7.8)

Then connect the ESP32 by USB, choose Tools, Board, "ESP32 Dev Module", select the COM port and press Upload. If the upload fails with "Failed to connect", hold the BOOT button for about two seconds while the IDE prints "Connecting...". Repeat for the second robot.

> Do not commit or share the sketches with real Wi-Fi credentials or the token inside them (observation F2).

11.6 Demonstration accounts

Table 28. Seeded accounts (passwords are set in `seed.py`)

Role	Username	Landing page
Administrator	`admin`	`/admin/overview`
Medicine Manager	`manager`	`/manager`
Emergency Ward / OT	`emergency`	`/ward/emergency`
General Ward 1	`ward1`	`/ward/general_1`
General Ward 2	`ward2`	`/ward/general_2`

The README and the setup guide give different passwords for the two ward accounts, so confirm what `seed.py` actually creates. Change every demonstration password before any real use. To test several roles on one laptop, use a normal tab for the manager, an incognito window for a ward and a second browser for the administrator, or open the site on a phone.

11.7 Troubleshooting

Table 29. Common problems

Symptom	Likely cause	What to do
TFT shows WIFI: OFF	Wrong SSID or password, or a 5 GHz network	Use a 2.4 GHz network and check the spelling of both values
Wi-Fi connects but no orders arrive	Wrong or missing server URL, laptop firewall, server not running	Use the laptop IPv4 address with `http://` and `:5000`; test from a phone; allow python.exe in the firewall
Only R1 gets no orders while R2 works	`SERVER_BASE_URL` in `bot_general.ino` lacks `http://`	Add the prefix (observation F1)
Upload fails: "Failed to connect to ESP32"	Board not in download mode	Hold BOOT while the IDE shows "Connecting..."
Serial monitor garbled, motors twitch at boot	ENA and ENB share pins with UART0	Move the enable pins (observation F4)
Robot does not follow the line, or runs off it	IR sensitivity, sensor height, or wrong logic level	Adjust the trimmers; set `LINE_DETECTED_VAL` for your modules; lower `MOTOR_SPEED`
Turns too short or too long	Timed pivot no longer matches battery and floor	Recalibrate `TURN_90_TIME_MS` and `TURN_180_TIME_MS`
Card scan does nothing	UID not in the sketch, wrong state, or reader on 5 V	Read the UID on the serial monitor; update the constants; power the RC522 from 3.3 V
Robot leaves before dispatch is clicked	Departure on READY_FOR_DISPATCH (observation F3)	Change the start condition if dispatch should gate departure

12. Testing and Verification

12.1 Automated backend tests

According to the review report, the supplied `tests/` suite was run on **22 October 2026** with `python -m pytest -q`. All **14 tests passed** in 7.78 s, using an isolated fixture database for each case. This is software evidence only. It is not a hospital deployment, a concurrency test or a robot trial.

Table 30. Executed backend regression tests

Area	Test file and cases	Result	Coverage limit
Access	`test_auth.py`: login and role boundaries; inactive user	2 passed	Test-client paths; no deployment security audit
Inventory	`test_inventory.py`: zero stock; reservation limit; single dispatch and duplicate guard; cancel and release	4 passed	Sequential fixtures; no simultaneous SQLite writers
Scheduling	`test_scheduler.py`: ward priority; same-ward FIFO	2 passed	Distinct timestamps only; manager selection not enforced
Fleet and workflow	`test_robot.py`: inactive or busy robot; dispatch gate; return release	3 passed	Software states only; no physical motion
Audit and interface	`test_audit.py`: audit actions; template response; robot API contract	3 passed	No ESP32 and no network-loss trial
Total		14 passed, 0 failed	No operational performance inference

12.2 Acceptance scenarios in the README

The README lists thirteen acceptance scenarios, A to M, as the intended coverage of the test suite:

Test	Scenario	Test	Scenario
A	Authentication and ward role isolation	H	Exactly-once inventory deduction on dispatch
B	Zero stock availability and order rejection	I	Idempotency and duplicate-request protection
C	Concurrent stock reservation limits	J	Safe reservation release on cancellation
D	Priority scheduling (Emergency, Ward 1, Ward 2)	K	Mandatory return confirmation before reassignment
E	FIFO tie-breaking	L	Mobile responsive viewport and controls
F	Inactive or busy robot assignment rejection	M	Security and operational audit-trail logging
G	Multi-step dispatch gate verification		

The review report states that the executed suite does not demonstrate true concurrency, strict manager queue compliance or the full ten-scenario field protocol, so scenario C in particular should be confirmed against the actual test code.

12.3 Field scenarios planned in the review report

Table 31. Planned scenarios S1 to S10 (no results recorded yet)

ID	Scenario	Evidence category	Current boundary
S1	Routine order with sufficient stock, full lifecycle	UI and backend	Planned
S2	Emergency order arrives while a routine order waits	UI and backend	Planned; record which order the manager actually picks

ID	Scenario	Evidence category	Current boundary
S3	Two same-priority orders, including equal timestamps	Backend	Planned
S4	Medicine shortage	UI and backend	Planned
S5	Reserve then cancel	UI and backend	Planned
S6	One robot unavailable	UI and backend	Planned
S7	Low reported battery	Backend	Planned; no assignment interlock exists
S8	Obstacle on the route	Simulation or hardware	Not executable: no obstacle sensing or route planner
S9	Robot communication failure	Failure injection, then hardware	Planned; no timeout watchdog exists
S10	Emergency request during an active delivery	Backend, later simulation or hardware	Planned; active preemption is not implemented

12.4 Proposed hardware bench tests

Now that firmware exists, the physical layer can be tested. These tests are proposals, not completed work. Record the result of each under the matching evidence category (hardware).

ID	Test	What to record
H1	IR sensors over black line, white floor and cross-line	Logic level, trimmer setting, correct `LINE_DETECTED_VAL`
H2	Straight and curved line following at the chosen `MOTOR_SPEED`	Completion, line losses, time over a measured length
H3	90° and 180° pivots, ten repeats each, fresh and half-charged battery	Angle error per repeat; updated timing constants
H4	Two-scan RFID sequence at the dock and at each ward	Correct card accepted, wrong card rejected, lid angles
H5	Poll, dispatch and event round trip with the real server	Status shown on both sides; confirms F1 and F3 behaviour
H6	Wi-Fi dropped during transit and during an event post	Recovery, lost events, server state afterwards (F13)
H7	Full runs: R1 to Ward 1, R1 to Ward 2, R2 to the OT	Delivery time, return time, stops, interventions
H8	Battery runtime under load	Runs per charge, voltage at cut-off

12.5 Measurements to collect

Table 32. Metric definitions (blank until measured)

Metric	Definition	Measured value
Order submission success rate	Valid orders stored divided by valid submission attempts	
Inventory consistency	Runs where P, R and the transaction records reconcile divided by checked runs	
Queue response time	Time from a request or setting change to the computed queue, for a stated data size	
Emergency waiting time	W = t_milestone − t_created for a named milestone (accept, assign or dispatch)	
Delivery time	Dispatch timestamp to verified handover	
Task completion rate	Returned or delivered missions divided by dispatched missions in a stated window	

Metric	Definition	Measured value
Robot utilisation	U = t_busy / t_available over a defined observation time	
Route time estimate	t_route = L / v + t_stop, with route length L, mean speed v and stop time t_stop	

12.6 Test evidence log

Test ID	Date	Tester	Category (UI, backend, simulation, hardware)	Observed result	Pass / fail

13. Literature and Research Context

The review report surveys work on hospital delivery robots, scheduling and multi-robot planning. Its conclusion is that these sources give design options and evaluation methods, not evidence that this prototype performs well. Reference numbers refer to the list at the end of this document.

Table 33. Literature and its relevance (condensed from the review report)

Topic	Sources	What it supports	Limit for this project
Hospital robotic courier simulation	Rossetti, Felder and Kumar [9]	Evaluating transport policy with turnaround and variability measures	Different fleet, demand and network; no result transfers to this prototype
Inventory and material handling	Bhosekar et al. [10]; Banks et al. [11]	Joint stock and transport experiments; simulation validation practice	Surgical instruments, not medicine requisition
Shortest-path search	Dijkstra [7]; Hart, Nilsson and Raphael [6]	Candidate route planners once a weighted corridor graph exists	No graph or planner in the project; single-robot
Multi-robot path planning	Silver [3]; Sharon et al. [4]; Švestka and Overmars [1]	Space-time reservation and conflict-based search for two robots	Needs a map, timing and collision model; proposed only
AMR planning and control	Fragapane et al. [5]	Separating task allocation, routing and fleet control	A review, not an implementation
Real-time scheduling	Liu and Layland [8]	Fixed versus dynamic priority concepts	Periodic-task bounds do not apply to irregular orders
Swarm and behaviour-based robotics	Bonabeau et al. [12]; Brambilla et al. [2]; Arkin [13]	Motivation for later decentralised study	The present system is centralised, not a swarm

13.1 Research gaps identified

- **Application level:** a correctly ordered queue display is not the same as a rule that guarantees the first eligible order is dispatched. Strict order would need an enforced transaction or an audited override.
- **Robotics level:** a verified build record, corridor map, positioning method, obstacle response and communication-failure recovery are still needed before a multi-robot pathfinder is chosen. Active emergency preemption also needs a safe payload-custody and rerouting protocol.
- **Evaluation level:** no measured delivery, battery or latency data exist yet. Results should be labelled by origin (UI, backend, simulation, hardware) and report workload, baseline and sample size.

The report also removed five earlier bibliography entries that could not be verified and replaced their topics with confirmed sources. It makes no claim of research novelty.

14. Limitations, Risks and Priorities

14.1 Current limitations

Table 34. Limitations by area

Area	Limitation
Evidence	No recorded delivery trials, timings, battery data or photographs of the built robots. Screenshots show an idle system. The 14 passing tests are backend only
Inventory and orders	SQLite reservation has no row lock; audit writes commit separately and can fail silently; the manager can bypass queue order; no failed-delivery state or restock workflow after dispatch
Fleet control	Manual robot choice; no battery-aware assignment; no timeout if a robot stops reporting; no robot-to-ward mapping enforced
Firmware	See observations F1 to F16: URL scheme, dispatch trigger, pin conflicts, constant battery value, identical card UIDs, blocking network calls, timed turns, no retry
Safety	No obstacle sensing, emergency stop or low-battery interlock. Medicine custody, recipient identity checks and clinical safety are outside the proven boundary
Security	Demonstration accounts and development secrets; plain HTTP; credentials and token written into firmware; audit log not proven tamper-resistant
Scope	No hospital information-system integration, drug identity verification at handover, door or lift control, or load-cell stock counting

14.2 Suggested order of fixes before a demonstration

1. Add `http://` to R1's server URL and confirm both robots can poll (F1).
2. Decide whether departure should wait for DISPATCHED, and change the firmware to match (F3).
3. Enforce R1 for general wards and R2 for the emergency ward in the backend, with a guard in the firmware (F10).
4. Read the real RFID cards and give the manager and ward cards distinct UIDs (F7).
5. Move ENA, ENB and IN4 off pins GPIO 1, 3 and 12, or confirm on the boards that they cause no problem (F4, F5).
6. Remove Wi-Fi credentials and the token from files before sharing them, and rotate them (F2).
7. Calibrate turn timings and speeds on the real track (F12).
8. Check how the server treats the heartbeat status values (F15) and reset the robot when an assignment disappears (F14).
9. Make the README, setup guide and review report agree (Chapter 16).
10. Run the scenarios in Section 12.3 and the bench tests in Section 12.4, and record the results in the evidence log.

15. Roadmap and Planned Work

The review report plans two tracks: a software track that validates inventory, orders and access, and a robotics track that is conditional on a documented physical platform. The firmware now makes the robotics track more concrete, but its later items (localisation, route planning, preemption) remain future work.

15.1 Software, inventory and deployment

Table 35. Software work plan

Work item	Dependency	Proposed action	Deliverable
Inventory consistency	Known stock and a reproducible test database	Test simultaneous orders, duplicate items, cancellation, dispatch and induced failures; adjust transactions or locking	Reconciliation tests; before and after P, R, A records
Queue policy	Agreed manager-override rule	Enforce the first eligible order or require an audited reason; compare fixed rank plus FIFO with a bounded-wait rule	Policy specification and tie tests
Access and secrets	Role matrix; deployment environment	Validate role checks on every mutation; remove demo credentials; HTTPS; token rotation	Permission test matrix and security review
Audit and recovery	Defined event set	Test log failure and partial commits; define post-dispatch incident entries	Event coverage report and recovery procedure
Telemetry and communication	HTTP client contract and heartbeat policy	Validate token handling and schemas; detect stale `last_seen`; add retries	API contract tests and integration logs
Usability	Representative ward and manager tasks	Observe cart, stock and dispatch tasks; improve confirmations and errors	Usability log and revised screens
Deployment	Institutional environment	Managed configuration, migrations, backup and restore, non-debug serving	Deployment guide

15.2 Robotics and safety (conditional)

Table 36. Robotics work plan

Work item	Dependency	Proposed action	Deliverable
Physical platform record	Confirmed robot count and parts	Document controller, driver, motors, battery, sensors, latch, wiring and limits; bench-test each unit	Bill of materials, drawings, bench record
Localisation	Surveyed area and sensors	Map stations and corridors; calibrate waypoint detection against ground truth	Versioned map and error report
Route and collision control	Map, timing and right-of-way rules	Compare a single-robot planner and a two-robot conflict policy; test obstacle stops	Route logs and safe-stop evidence
Battery-aware dispatch	Measured capacity	Define a reserve for mission plus return; enforce an assignment guard	Calibrated threshold and test results
Emergency preemption	Operational approval, routes, stop and restart controls	Specify whether R2 serves first; if interruption is allowed, stop at a safe node, secure the payload, replan and resume or reassign	State-machine design and S10 trials
Communication and mission recovery	Heartbeat timeout and custody policy	Fail-safe behaviour on link loss; manager alert; reconcile after reconnection	Fault-injection logs (S9)
Physical evaluation	Built platform and safety protocol	Repeat routine, emergency, obstacle and return trials with synchronised clocks	Dataset of time, path, battery, completion and incidents

15.3 Proposed 20-week timeline

This is a proposed relative schedule counted from an agreed start. Align it with the actual submission deadline. The robotics rows are conditional on a confirmed build.

Table 37. Proposed Gantt schedule and milestones

Phase and milestone	Dependency	W1-4	W5-8	W9-12	W13-16	W17-20
Baseline inventory and test data (M1: reproducible environment)	Source revision and seed	■				
Backend and UI scenario tests, stock reconciliation (M2: verified software)	M1	■	■			
Queue, audit, access and deployment hardening (M3: reviewed workflow)	M1, initial findings		■	■		
Physical build and telemetry interface (M4: bench evidence, conditional)	Component and build confirmation		■	■		
Map, localisation, two-robot route safety (M5: controlled course, conditional)	M4, site model			■	■	
Battery, loss-of-link and preemption design (M6: safety review, conditional)	M4 to M5, custody policy				■	■
Integrated trials, analysis, final evidence (M7: reviewable report)	M2 to M3; M5 to M6 if hardware				■	■

A missed gate should change the stated scope of the project rather than lead to estimated values being entered as results. If physical integration cannot be completed by the deadline, the outcome should be limited to the tested software workflow and a clearly defined simulation, with routing and active preemption kept as future work.

16. Reconciling the Source Documents

The project files were written at different times and disagree in several places. The table lists each difference and a suggested resolution, so the final documents tell one consistent story.

Table 38. Differences between the project documents

Topic	What the files say	Suggested resolution
Test count	README: 13 scenarios (A to M). Setup guide and review report: 14 tests, all passing on 22 October 2026	Say "14 tests covering scenarios A to M" and update the README
Ward passwords	README: one password for both wards. Setup guide: different passwords for ward 1 and ward 2	Check `seed.py`, then correct whichever document is wrong
Ward page paths	README and review report: `/ward/<code>`. Setup guide: `/room/<code>`	Use `/ward/<code>` and fix the setup guide
Robot roles	README: R1 and R2 as a generic pair. Setup guide and firmware: R1 for general wards, R2 for emergency	Document the dedicated mapping and enforce it in the backend (F10)
Dispatch step	README: "Complete & Dispatch", no RFID. Setup guide: "Dispatch Robot" plus RFID scans on the robot	Describe the web action and the on-robot scans together
Robot integration	README: ESP32 integration is "future". Setup guide and firmware: it exists. Review report: no firmware supplied	Update the README and the report status tables (Table 1.1 and Section 4.2.10 of the report)
Arrival detection	README: arrival when the robot detects the destination RFID tag. Firmware: both IR sensors on a cross-line; `/rfid_node` never called	Describe IR marker detection (F11)
IR sensor type	R1 header comment: 8-channel, proportional. Setup guide and code: two digital sensors, on/off steering	Fix the comment (F8)
Departure trigger	Setup guide: robot starts on DISPATCHED. Firmware: READY_FOR_DISPATCH or DISPATCHED	Decide and align (F3)
Project title	Certificate and brief versus review report cover title differ	Choose one title for all documents

Appendix A. Glossary

Term	Meaning
AMR	Autonomous mobile robot
Audit trail	Time-stamped record of actions with actor, entity and details
Bang-bang control	On/off steering that turns left, right or straight depending on two sensors, with no proportional correction
CSRF	Cross-site request forgery; Flask-WTF protects browser forms against it
ESP32	Dual-core microcontroller with built-in Wi-Fi and Bluetooth
FIFO	First in, first out
GPIO	General-purpose input/output pin
IR sensor	Infrared reflective sensor used to tell a black line from a light floor
L298N	Dual H-bridge DC motor driver
LEDC	ESP32 PWM peripheral used for motor speed control
Preemption	Interrupting work already in progress to serve something more urgent
PWM	Pulse-width modulation
RBAC	Role-based access control
RFID	Radio-frequency identification; here an RC522 reader and 13.56 MHz cards
SPI	Serial peripheral interface bus shared by the TFT and the RFID reader
Strapping pin	ESP32 pin whose level at reset selects a boot option
TFT	Thin-film transistor colour display (ILI9341, 320 × 240)
UID	Unique identifier read from an RFID card

References

> The reference list is carried over from the team's review report. It was not independently re-checked when this document was compiled, and I cannot search a bibliographic database from here. Please verify authors, venues, years and DOIs before submission.

[1] P. Švestka and M. H. Overmars, "Coordinated path planning for multiple robots," Robotics and Autonomous Systems, vol. 23, no. 3, pp. 125-152, 1998.

[2] M. Brambilla, E. Ferrante, M. Birattari, and M. Dorigo, "Swarm robotics: A review from the swarm engineering perspective," Swarm Intelligence, vol. 7, pp. 1-41, 2013, doi: 10.1007/s11721-012-0075-2.

[3] D. Silver, "Cooperative pathfinding," in Proc. 1st AAAI Conf. Artificial Intelligence and Interactive Digital Entertainment (AIIDE), pp. 117-122, 2005, doi: 10.1609/aiide.v1i1.18726.

[4] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," Artificial Intelligence, vol. 219, pp. 40-66, 2015.

[5] G. Fragapane, R. de Koster, F. Sgarbossa, and J. O. Strandhagen, "Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda," European Journal of Operational Research, vol. 294, no. 2, pp. 405-426, 2021, doi: 10.1016/j.ejor.2021.01.019.

[6] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[7] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269-271, 1959, doi: 10.1007/BF01386390.

[8] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," Journal of the ACM, vol. 20, no. 1, pp. 46-61, 1973, doi: 10.1145/321738.321743.

[9] M. D. Rossetti, R. A. Felder, and A. Kumar, "Simulation of robotic courier deliveries in hospital distribution services," Health Care Management Science, vol. 3, no. 3, pp. 201-213, 2000, doi: 10.1023/A:1019049609350.

[10] A. Bhosekar, S. D. Ekşioğlu, T. Işık, and R. Allen, "A discrete event simulation model for coordinating inventory management and material handling in hospitals," Annals of Operations Research, vol. 320, no. 2, pp. 603-630, 2023, doi: 10.1007/s10479-020-03865-5.

[11] J. Banks, J. S. Carson II, B. L. Nelson, and D. M. Nicol, Discrete-Event System Simulation, 5th ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[12] E. Bonabeau, M. Dorigo, and G. Theraulaz, Swarm Intelligence: From Natural to Artificial Systems. New York, NY, USA: Oxford University Press, 1999.

[13] R. C. Arkin, Behavior-Based Robotics. Cambridge, MA, USA: MIT Press, 1998.